

RichTerm - Bedienungsanleitung

Chat mit Claude Code (oder einer lokalen KI) mit gerenderten Formeln, Diagrammen und Animationen, plus Terminal, in einem Fenster. Stand: Oktober 2026.

- | | |
|---|--|
| 1. Was RichTerm ist | 8. Berechtigungen |
| 2. Starten | 9. Das Terminal und der Befehl <code>rt</code> |
| 3. Die Oberfläche | 10. Tastenkürzel |
| 4. Das Profil: <code>richterm.md</code> | 11. Typische Arbeitsabläufe |
| 5. Modelle wählen und laden | 12. Probleme und Lösungen |
| 6. Unterlagen: der Ordner <code>rag/</code> | 13. Dateien im Überblick |
| 7. Gedächtnis und Sitzungen | |

1. Was RichTerm ist

RichTerm ist ein Fenster mit zwei Arten von Tabs:

- **Chat:** Du schreibst unten, die KI antwortet oben. Die Antworten werden gerendert wie in einem Browser-Chat: Markdown, LaTeX-Formeln in Buchqualität, Tabellen, Mermaid-Diagramme, Bilder und HTML-Blöcke als Live-Vorschau (Animationen, interaktive Grafiken).
- **Terminal:** ein vollwertiges Terminal (dieselbe Engine wie GNOME Terminal), beliebig viele Tabs.

Die KI arbeitet in einem **Arbeitsordner**. Mit Claude Code als Backend kann sie die Dateien darin lesen, Code ausführen und Dateien schreiben, etwa Vorlesungsfolien lesen und dazu Fragen beantworten. In jedem Ordner liegen zwei Dateien, die RichTerm selbst anlegt:

`richterm.md` (das Profil: Rolle, Verhalten, Modell) und `richterm-verlauf.md` (das Gedächtnis).

Es ist nichts installiert worden: RichTerm besteht aus Python und den Bibliotheken, die Linux Mint ohnehin mitbringt. Das Projekt liegt unter `~/Desktop/RichTerm` und ist ein Git-Repository.

2. Starten

Zum ersten Mal (neuer Rechner)

```
git clone https://github.com/DavyvSz/RichTerm.git
cd RichTerm
./richterm
```

Beim ersten Start richtet sich RichTerm selbst ein: Es prüft die Systempakete und installiert fehlende (fragt nach dem Administrator-Passwort), legt die Befehle `richterm` und `rt` an,

Menü- und Desktop-Einträge, und bietet an, Claude Code und ein lokales Ollama zu installieren. Ist alles da, startet es einfach.

macOS: Browser-Modus

Auf dem Mac gibt es kein natives Fenster (die Terminal- und Browser-Engine des Linux-Fensters existieren dort nicht). Stattdessen startet `richterm` den **Browser-Modus**: Python liefert den Chat aus, und er öffnet sich als eigenes Fenster in Chrome, Chromium oder Edge (ohne Adressleiste), sonst im Standardbrowser. Alles außer den Terminal-Tabs funktioniert gleich. Den Arbeitsordner gibst du beim Start an (`richterm ~/Uni/ML`) oder über den Ordnerknopf (Pfad eintippen). RichTerm beendet sich von selbst etwa 90 Sekunden nach dem Schließen des Browserfensters. Unter Linux erzwingt `richterm --web` denselben Modus.

Im aktuellen Ordner (der übliche Weg)

```
cd ~/Uni/Maschinelles_Lernen    # Ordner mit Folien, Übungen, Notizen
richterm                        # RichTerm öffnet sich mit diesem Ordner
```

Das Terminal ist sofort wieder frei und darf geschlossen werden. Mit Pfad geht es von überall: `richterm ~/Uni/ML`. Jeder Aufruf öffnet ein eigenes Fenster; du kannst also ein RichTerm für die Vorlesung und eins für ein Projekt gleichzeitig offen haben.

Weitere Wege

- **Dateimanager (Nemo):** Rechtsklick auf einen Ordner oder in einen Ordner hinein, „**RichTerm hier öffnen**“.
- **Desktop-Icon / Anwendungsmenü:** startet im Home-Ordner; den Ordner wechselst du danach über den Ordnerknopf oben links.

Beim ersten Start in einem Ordner werden `richterm.md` und `richterm-verlauf.md` angelegt. Die Statuszeile unter dem Eingabefeld weist darauf hin. Fülle das Profil aus (Abschnitt 4), bevor du ernsthaft arbeitest.

3. Die Oberfläche

Element	Funktion
Ordnerknopf (oben links)	Zeigt den Arbeitsordner; Klick öffnet die Ordnerwahl (im Browser-Modus: Pfad eintippen). Ein Wechsel startet eine neue Sitzung.
Modellmenü	Welche KI antwortet: Claude-Modelle, lokale Modelle, „Lokales Modell herunterladen ...“ (Abschnitt 5). Gilt für diesen Ordner.
Berechtigungen	Fragen / Dateien bearbeiten erlauben / Alles erlauben (Abschnitt 7). Gesperrt, wenn das Profil es festlegt.
Profil	Öffnet <code>richterm.md</code> im Editor.
Neu laden	Liest das Profil neu ein (nach dem Bearbeiten). Gilt ab der nächsten Nachricht.
* / ☾	Hell / dunkel.
Neuer Chat	Beendet die laufende Sitzung und beginnt frisch (Abschnitt 6).
Eingabefeld	<code>Enter</code> sendet, <code>Shift+Enter</code> macht einen Zeilenumbruch. ■ bricht eine laufende Antwort ab.
Statuszeile	Über dem Eingabefeld: Ordner, Sitzung, Dauer und Kosten der letzten Antwort, Hinweise.

Anhänge: Bilder und Dateien

Bilder (Screenshot einer Folie, Foto vom Tafelanschrieb), PDFs und andere Dateien kannst du an eine Frage anhängen: **in das Fenster ziehen**, mit `Ctrl+V` **einfügen** (z. B. einen Screenshot) oder über die -Schaltfläche wählen. Die Anhänge erscheinen als kleine Chips über dem Eingabefeld (× entfernt sie) und werden mit der nächsten Nachricht gesendet. Claude sieht Bilder direkt (es kann also eine handschriftliche Rechnung lesen und korrigieren); andere Dateien liest es mit seinem Read-Werkzeug. Eingefügte Dateien werden im Arbeitsordner unter `richterm-anhang/` abgelegt; aus dem Dateimanager gezogene Dateien bleiben, wo sie sind.

Unter jeder Antwort

- **Einfacher · Kürzer · Beispiel · Abfragen:** Schnellaktionen. Ein Klick stellt die passende Folgefrage („Abfragen“ lässt dich die KI mit Prüfungsfragen abfragen).
- **Lernzettel speichern:** legt die Antwort als Markdown-Datei (mit Frage, Formeln und Code) in `lernzettel/` im Arbeitsordner ab. So entsteht aus guten Erklärungen eine eigene Sammlung.
- **Kopieren** an jedem Codeblock kopiert den ganzen Code.

In den Antworten

- **Graue Zeilen** sind Werkzeugaufrufe (Datei gelesen, Befehl ausgeführt). Ein Klick zeigt Details und das Ergebnis.

- **Live-Vorschau** bei HTML-Blöcken: *Code* zeigt den Quelltext, *Neu laden* startet die Animation neu, *Im Browser* öffnet die Seite groß.
- **Berechtigungsfrage**: ein Kasten mit *Erlauben* / *Ablehnen*, wenn die KI ein Werkzeug benutzen will, das nicht freigegeben ist.
-   /  ändert die Schriftgröße des Chats.

4. Das Profil: richterm.md

Das Profil ist die wichtigste Datei. Es wird bei jedem Start gelesen und der KI als **verbindliche Anweisung** mitgegeben, egal ob Claude oder eine lokale KI antwortet. Es gilt nur für diesen Ordner. So verhält sich die KI im Vorlesungsordner wie ein Tutor und im Projektordner wie ein Entwickler.

Oben: Einstellungen

Feld	Werte	Bedeutung
<code>backend</code> , <code>model</code>	werden vom Modellmenü gesetzt	Nicht von Hand nötig. <code>claude</code> = Claude Code mit Dateizugriff; <code>command</code> = andere KI, nur Chat.
<code>command</code>	z. B. <code>ollama run {model}</code>	Nur für <code>command . {model}</code> wird ersetzt.
<code>permissions</code>	<code>default</code> · <code>acceptEdits</code> · <code>bypassPermissions</code>	<code>default</code> = Kopfzeile entscheidet; sonst festgelegt und gesperrt.
<code>allowed_tools</code>	z. B. <code>Read Grep Glob</code>	Nur diese Werkzeuge: hier reines Lesen, nichts wird geändert.
<code>disallowed_tools</code>	z. B. <code>WebSearch "Bash(rm *)"</code>	Diese Werkzeuge verbieten.
<code>language</code>	z. B. <code>Deutsch</code>	Sprache der Antworten.
<code>history</code>	<code>true</code> · <code>false</code>	Gedächtnis (Verlaufsdatei) an oder aus.
<code>context_chars</code>	Zahl, Standard 20000	Wie viel Verlauf beim Start mitgegeben wird. Claude: 20000-60000; kleine lokale Modelle: 4000-8000.

Darunter: Anweisungen in normaler Sprache

Die Vorlage enthält sieben Abschnitte mit Beispielen. Schreib sie einfach um, lösche, was nicht passt:

- **Rolle:** Tutor, Entwickler, Reviewer, Prüfungsvorbereiter, Schreibassistent ...
- **Themengebiet und Ziel:** „Vorlesung ML, Klausur im Februar, Schwerpunkt neuronale Netze.“
- **Was in diesem Ordner liegt:** „folien/ = PDFs, uebungen/ = Übungsblätter, notizen.md = meine Notizen.“
- **Verhalten:** schrittweise erklären, Rückfragen stellen, Quellen aus dem Ordner nennen.
- **Genauigkeit und Kreativität:** keine erfundenen Fakten; „steht in den Unterlagen“ vs. „Einschätzung“ trennen.
- **Antwortformat:** LaTeX-Formeln, nummerierte Herleitungen, Diagramme, Länge.
- **Nicht erlaubt:** nichts löschen, Folien nur lesen ...

Nach jeder Änderung: im Chat auf *Neu laden* klicken. Die Änderung gilt ab der nächsten Nachricht (es wird eine neue Sitzung gestartet; der Verlauf bleibt erhalten).

5. Modelle wählen und laden

Oben im Chat gibt es das **Modellmenü**. Dort stehen alle Claude-Modelle und alle lokalen Modelle, die auf deinem Rechner liegen. **Auswählen, fertig**. Die Wahl gilt für diesen Ordner und ab der nächsten Nachricht; RichTerm trägt sie selbst in `richterm.md` ein. Du musst keine Datei bearbeiten.

Menüeintrag	Was es ist
Claude (Standard) / Opus / Sonnet / Haiku	Claude Code. Liest deine Dateien, führt Code aus, schreibt Dateien. Opus = am stärksten, Sonnet = ausgewogen, Haiku = schnell und günstig. Kosten pro Antwort stehen in der Statuszeile.
Lokal (Ollama): z. B. qwen2.5:7b (4.7 GB)	Läuft komplett auf deinem Rechner: kostenlos, nichts verlässt den PC. Kann keine Dateien lesen, nur chatten. Langsamer und einfacher als Claude.
Lokales Modell herunterladen ...	Ein Feld erscheint: Modellnamen eintippen (z. B. <code>qwen2.5:7b</code>), <i>Laden</i> klicken. Der Fortschritt steht in der Statuszeile; danach ist das Modell im Menü und ausgewählt.

Ein neues lokales Modell holen, Schritt für Schritt

1. Im Modellmenü „**Lokales Modell herunterladen ...**“ wählen.
2. Namen eintippen, z. B. `qwen2.5:7b`, und auf **Laden** klicken. Namen findest du auf `ollama.com/library`.
3. Warten (4,7 GB dauern ein paar Minuten; die Statuszeile zeigt Prozent). Das Modell ist danach automatisch ausgewählt.
4. Frage stellen.

Zurück zu Claude: im Menü wieder ein Claude-Modell wählen.

Lokales Modell	Größe	Eignung auf deinem Laptop (CPU, 31 GB RAM)
<code>qwen2.5:0.5b</code>	0,4 GB	Nur zum Testen; antwortet in Sekunden, ignoriert Kontext oft.
<code>qwen2.5:3b</code>	1,9 GB	Schnell, einfaches Niveau.
<code>qwen2.5:7b</code>	4,7 GB	Beste Mischung aus Deutsch, Mathe und Code; 4-8 Wörter/s.
<code>llama3.1:8b</code>	4,9 GB	Solide Allzweck-Wahl.
<code>qwen2.5:14b</code>	9 GB	Besser, aber nur 1-3 Wörter/s.

Hintergrund (musst du nicht wissen): Lokale Modelle werden über die Ollama-API angesprochen (Streaming, Gesprächsverlauf innerhalb der Sitzung). Bildfähige lokale Modelle (z. B. `llava`, `qwen2.5vl`) bekommen angehängte Bilder ebenfalls. RichTerm bringt ein eigenes Ollama mit (unter `~/.local/share/ollama`, Port 11435) und startet es bei Bedarf selbst. Im RichTerm-Terminal geht auch die Kommandozeile: `ollama list`, `ollama pull name`, `ollama rm name`. Andere KIs mit Kommandozeile lassen sich im Profil über `backend: command` und `command: ...` anbinden.

6. Unterlagen: der Ordner rag/

Das ist die wichtigste Funktion fürs Lernen mit Vorlesungsfolien. Lege im Arbeitsordner einen Ordner `rag/` an und kopiere alles hinein, womit die KI arbeiten soll: Folien und Skripte als PDF, Übungsblätter, eigene Notizen (Markdown oder Text), auch DOCX, HTML und Code.

- RichTerm zerlegt die Dateien automatisch in Abschnitte (PDFs **seitenweise**) und legt einen Index an (`rag/.richterm-index.json`). Neue oder geänderte Dateien werden beim nächsten Start oder bei der nächsten Frage erkannt.
- Zu jeder Frage sucht RichTerm die passendsten Abschnitte heraus und gibt sie der KI mit, zusammen mit der Anweisung, sich darauf zu stützen und die **Quelle zu nennen** (Datei und Seite). Die Antwort sagt dann z. B. „laut vorlesung_03.pdf S. 4“.
- Unter deiner Frage steht „📖 Unterlagen: ...“ mit den verwendeten Dateien und Seiten.
- Das funktioniert mit jedem Modell, auch mit lokalen. Die Dateien bleiben im Ordner; es werden nur die passenden Textausschnitte an das Modell gegeben.

Wie die Suche arbeitet

Die Suche ist hybrid: Sie findet Stichwörter (BM25) **und** Bedeutung. Für Letzteres lädt RichTerm beim ersten Mal automatisch ein kleines Einbettungsmodell über Ollama (`embeddinggemma` , 620 MB, mehrsprachig, läuft offline). Damit findet „Wie groß ist die Schrittweite?“ auch eine Folie, auf der „Lernrate“ steht, und Fragen ohne Bezug zu den Unterlagen („Hauptstadt von Peru“) liefern keine Treffer, statt irgendetwas hineinzuzwingen. Ist kein Ollama da, bleibt die Stichwortsuche.

Profil-Einstellung	Wirkung
<code>rag: false</code>	Unterlagen-Suche aus.
<code>rag_chunks:</code> <code>6</code>	Wie viele Abschnitte pro Frage mitgehen.
<code>rag_strict:</code> <code>true</code>	Strenger Modus: Die KI antwortet nur aus den Unterlagen, jede Aussage mit Quelle; steht etwas nicht drin, sagt sie das, statt eigenes Wissen zu ergänzen. Gut für „Was hat der Prof gesagt?“.

Ganze Datei mitgeben: Schreib `@vorlesung_03.pdf` (oder einen Anfang des Namens, `@vorlesung_03`) in die Frage, dann bekommt die KI die komplette Datei statt nur Treffer. Das ist der Weg für „Fasse die Vorlesung zusammen“. Bei Claude passen sehr lange Dateien hinein, bei lokalen Modellen wird nach etwa 12 000 Zeichen gekürzt.

Praktisch: Die Zeile „ Unterlagen: ...“ unter deiner Frage zeigt, was mitgeschickt wurde. Fehlt sie, wurde nichts Passendes gefunden, und die Antwort kommt nicht aus deinen Unterlagen. Für PDFs wird `pdftotext` gebraucht (Linux: Paket `poppler-utils`, meist vorhanden; macOS: `brew install poppler`).

7. Gedächtnis und Sitzungen

Es gibt zwei Ebenen, die zusammenarbeiten:

Claude-Code-Sitzung

Claude Code speichert jedes Gespräch selbst (unter `~/.claude/projects/...`), mit allem, was es gelesen und getan hat. RichTerm merkt sich pro Ordner die letzte Sitzung und **setzt sie beim nächsten Start automatisch fort** (Statuszeile: „Letzte Claude-Sitzung wird fortgesetzt“). *Neuer Chat* beendet das und beginnt frisch.

Die Verlaufsdatei `richterm-verlauf.md`

Das modellunabhängige Gedächtnis im Ordner. Jede Frage und Antwort wird angehängt, mit Zeit, Modell und LaTeX-Formeln. Beim Start einer neuen Sitzung, also nach einem Modell- oder Backend-Wechsel oder nach *Neuer Chat*, bekommt die KI die **Zusammenfassung** oben in der Datei und den **jüngsten Verlauf** mit. Nur darüber weiß Sonnet, was du mit Haiku besprochen hast, oder ein lokales Modell, was Claude erklärt hat. Beim Start zeigt der Chat die letzten Einträge grau an.

Automatische Verdichtung: Wird der Rohverlauf länger als das Budget (`context_chars`), fasst das Modell den älteren Teil zur Zusammenfassung zusammen. Das Original wandert nach `richterm-verlauf.archiv.md`; nichts geht verloren.

Beide Dateien darfst du bearbeiten: die Zusammenfassung kürzen, Wichtiges ergänzen, Falsches streichen. `history: false` im Profil schaltet das Gedächtnis ab.

8. Berechtigungen

Mit Claude Code kann die KI Werkzeuge benutzen: Dateien lesen und schreiben, Befehle ausführen, im Web suchen. Was ohne Nachfrage erlaubt ist, steuert der Berechtigungsmodus:

Modus	Verhalten	Wann sinnvoll
Fragen (<code>default</code>)	Lesen geht sofort; Schreiben und Befehle werden im Chat erfragt (Erlauben / Ablehnen).	Lernen, Unterlagen lesen, alles, wo nichts verändert werden soll.
Dateien bearbeiten erlauben (<code>acceptEdits</code>)	Dateien ändern ohne Nachfrage; Befehle werden weiter erfragt.	Programmieren im Projektordner.
Alles erlauben (<code>bypassPermissions</code>)	Keine Nachfragen.	Nur in Ordnern, in denen nichts kaputtgehen kann.

Feiner geht es mit `allowed_tools` / `disallowed_tools` im Profil, etwa `allowed_tools: Read Grep Glob` für einen Ordner, der nur gelesen werden darf.

9. Das Terminal und der Befehl `rt`

`Ctrl+Shift+T` öffnet ein Terminal im Arbeitsordner. Darin sind `rt`, `ollama` und `claude` direkt verfügbar. Mit `rt` schickst du Inhalte aus dem Terminal in den Chat:

```
rt ask 'Erkläre mir Backpropagation'    # Frage an den Chat
rt latex '\frac{1}{1+e^{-z}}'           # Formel als Karte im Chat
rt md notizen.md                        # Markdown-Datei (mit $...$-Formeln) anzeigen
rt image plot.png                      # Bild anzeigen (auch GIF)
rt html seite.html                     # HTML-Seite mit Animation einbetten
rt mermaid 'graph LR; A-->B'           # Diagramm
rt clear                               # Chat-Karten entfernen
rt --help                              # alle Möglichkeiten
```

`rt` funktioniert auch in jedem anderen Terminal, solange RichTerm läuft. Startest du `claude` in einem RichTerm-Terminal, weiß es durch `~/claude/CLAUDE.md`, dass es Formeln und Bilder mit `rt` in den Chat-Tab schicken kann.

10. Tastenkürzel

Taste	Wirkung
<code>Enter</code> / <code>Shift+Enter</code>	Senden / Zeilenumbruch (Chat)
<code>Ctrl</code> + <code>+</code> / <code>-</code>	Chat-Schrift größer / kleiner
<code>Ctrl+Shift+T</code>	Neues Terminal im Arbeitsordner
<code>Ctrl+Shift+W</code>	Terminal schließen
<code>Ctrl+PgUp</code> / <code>Ctrl+PgDn</code>	Tab wechseln
<code>Ctrl+Shift+Enter</code>	Zum Chat springen
<code>Ctrl+Shift+C</code> / <code>Ctrl+Shift+V</code>	Kopieren / Einfügen im Terminal (Rechtsklick fügt auch ein)
<code>Ctrl+Shift++</code> / <code>Ctrl+Shift+-</code>	Terminal-Schrift größer / kleiner
<code>Ctrl</code> + Klick auf Link	Link im Browser öffnen (Terminal)

11. Typische Arbeitsabläufe

Lernen mit Vorlesungsfolien

1. Ordner anlegen, z. B. `~/Uni/ML`, darin `rag/` mit Folien, Skripten und Übungen (Abschnitt 6).
2. `cd ~/Uni/ML` und `richterm`.
3. *Profil* klicken: Rolle „Tutor“, Thema, Ordnerstruktur, „Quellen nennen“, Berechtigungen auf Fragen lassen. *Neu laden*.
4. Fragen: „Fass Folie 10–15 aus `vorlesung_03.pdf` zusammen und leite die Formel her.“ Claude liest das PDF und antwortet mit Formeln.
5. Am nächsten Tag wieder `richterm`: Die Sitzung wird fortgesetzt, der Verlauf steht im Chat.

Programmieren

1. `richterm ~/Projekte/mein-projekt`.
2. Profil: Rolle „Entwickler“, `permissions: acceptEdits`, ggf. Verbote (nichts löschen, keine Git-Pushes).
3. Aufgaben stellen; Werkzeugaufrufe erscheinen als graue Zeilen, Klick zeigt Details.

Kostenlos und privat mit lokalem Modell

1. Modellmenü → „Lokales Modell herunterladen ...“ → `qwen2.5:7b` → Laden (einmalig).
2. Das Modell ist danach ausgewählt; sonst im Menü wählen.
3. Fragen stellen; Antworten dauern länger und sind einfacher, aber nichts verlässt den Rechner.

12. Probleme und Lösungen

Problem	Lösung
Desktop-Icon tut nichts	Rechtsklick auf das Icon, „Starten erlauben“. Oder im Terminal <code>richterm</code> .
„Der Befehl claude wurde nicht gefunden“	Claude Code ist nicht installiert oder nicht im PATH. Im Terminal <code>claude --version</code> prüfen.
Lokales Modell antwortet nicht oder ewig	Im RichTerm-Terminal <code>ollama list</code> und <code>ollama ps</code> prüfen. Läuft ein alter System-Ollama auf 11434, stört er nicht; RichTerm benutzt Port 11435. Notfalls <code>pkill -f "ollama serve"</code> für den eigenen Dienst; er startet beim nächsten Mal neu.
Antwort wird abgeschnitten / Prozess beendet	Statuszeile lesen; <i>Neuer Chat</i> starten. Der Verlauf bleibt in <code>richterm-verlauf.md</code> .
Profiländerung wirkt nicht	<i>Neu laden</i> klicken; gilt ab der nächsten Nachricht. Prüfen, ob das Feld oben (Einstellungen) oder unten (Text) gemeint ist.
Berechtigungen in der Kopfzeile gesperrt	Das Profil legt sie fest (<code>permissions:</code>). Dort <code>default</code> eintragen, dann entscheidet die Kopfzeile. Bei lokalen Modellen gibt es keine Berechtigungen, weil sie keine Werkzeuge haben.
Formeln werden nicht gerendert	Die KI muss LaTeX mit <code>\$...\$</code> schreiben; das steht im Profil. Bei lokalen Modellen hilft ein Hinweis in der Frage.
Kosten im Blick behalten	Die Statuszeile zeigt Dauer und Kosten jeder Antwort. Haiku ist am günstigsten.

13. Dateien im Überblick

Ort	Inhalt
<code>~/Desktop/RichTerm/</code>	Das Programm (Git-Repository): <code>richterm.py</code> , <code>chat/</code> , <code>bin/rt</code> , <code>vendor/</code> , <code>profile_template.md</code> , <code>start.sh</code> , <code>README.md</code> , <code>docs/</code> (diese Anleitung).
<code><Arbeitsordner>/richterm.md</code>	Profil für diesen Ordner.
<code><Arbeitsordner>/richterm-verlauf.md</code>	Gedächtnis; <code>richterm-verlauf.archiv.md</code> = verdichtete Rohteile.
<code><Arbeitsordner>/rag/</code>	Unterlagen, aus denen die KI schöpft (plus Index <code>.richterm-index.json</code>).
<code><Arbeitsordner>/lernzettel/</code>	Gespeicherte Antworten (Markdown).
<code><Arbeitsordner>/richterm-anhang/</code>	Eingefügte Bilder und Dateien.
<code>~/.config/richterm.json</code>	Fenstergröße, Schrift, letzter Ordner, gemerkte Claude-Sitzungen pro Ordner.
<code>~/.cache/richterm/html/</code>	Abgelegte HTML-Vorschauen (kann gelöscht werden).
<code>~/.local/share/ollama/</code>	Eigenes Ollama (<code>dist/</code>) und Modelle (<code>models/</code>).
<code>~/.local/bin/richterm</code> , <code>~/.local/bin/rt</code>	Die Befehle (Verweise ins Projekt).
<code>~/.claude/CLAUDE.md</code>	Globale Hinweise für Claude Code (u. a. zu <code>rt</code>).